

Dokumentace procesu DQ10000 v BigQuery

Sběr, audit a úklid DQ tabulek ze stg_lnd do daq_data

Položka	Hodnota
Projekt	o2czed1
Zdrojový dataset	stg_lnd
Cílový dataset	daq_data
Region	europa-west4
Pattern zdrojových tabulek	__dq10000__
Cílová tabulka	daq_data.dq10000_all
Hlavní procedura	daq_data.sp_collect_dq10000
Run audit	daq_data.dq10000_load_audit
Table audit	daq_data.dq10000_load_table_audit

Účel dokumentu

Tento dokument popisuje účel, logiku a provozní kontroly BigQuery procedury pro zpracování DQ10000 tabulek. Má sloužit kolegům jako rychlé vysvětlení, co proces dělá, proč existuje, jaké objekty používá, co se audituje a jaké jsou další plánované kroky.

1. Kontext a důvod řešení

V rámci datového zpracování v BigQuery dochází při validaci vstupních souborů k odmítnutí části dat. Tyto odmítnuté záznamy se ukládají jako DQ záznamy do landing vrstvy. V datasetu stg_lnd vznikají technické tabulky, jejichž název obsahuje pattern __dq10000__.

Tyto tabulky představují auditní stopu záznamů, které byly při zpracování zahozeny nebo označeny jako nevalidní. Obsahují například zdrojový soubor, číslo řádku, kód chyby, vstupní port, sanitizovanou hodnotu a původní řádek.

Původní stav byl takový, že v landing datasetu vznikalo větší množství samostatných DQ tabulek. Některé obsahovaly data, jiné byly prázdné. Cílem procesu je neprázdné a bezpečně dokončené DQ tabulky sloučit do jedné centrální tabulky v daq_data a následně zdrojové DQ tabulky uklidit.

2. Co proces řeší

- Vyhledá DQ tabulky v datasetu stg_lnd podle patternu __dq10000__.
- Pracuje pouze s fyzickými tabulkami typu BASE TABLE.
- Zpracuje jen tabulky starší než 12 hodin, aby se nesahalo na tabulky, které mohou být ještě plněné ingestem.
- Do slévání bere pouze tabulky s daty, tedy total_rows > 0.
- Dynamicky sestaví jeden UNION ALL přes všechny vhodné tabulky.
- Vloží data do cílové tabulky daq_data.dq10000_all.

- Zapiše hlavní audit běhu i detailní audit jednotlivých zdrojových tabulek.
- Po ověření počtů řádků provede DROP pouze těch zdrojových tabulek, které byly úspěšně načteny.

3. Používané objekty

Objekt	Typ	Popis
o2czed1.stg_lnd	Dataset	Landing vrstva, ve které vznikají zdrojové DQ tabulky obsahující __dq10000__.
o2czed1.daq_data	Dataset	Cílový dataset pro agregovaná DQ data, audit běhů a audit zdrojových tabulek.
daq_data.dq10000_all	Tabulka	Centrální tabulka se sloučenými DQ10000 záznamy.
daq_data.dq10000_load_audit	Tabulka	Audit běhů procedury. Jeden řádek odpovídá jednomu spuštění procedury.
daq_data.dq10000_load_table_audit	Tabulka	Detailní audit zdrojových tabulek zpracovaných v konkrétním běhu.
daq_data.sp_collect_dq10000	Stored procedure	Procedura, která provádí sběr dat, audit a po úspěšném načtení také DROP zdrojových DQ tabulek.

4. Struktura cílové tabulky dq10000_all

Cílová tabulka obsahuje původních 9 DQ sloupců a na konci technické sloupce. Technické sloupce jsou záměrně umístěné až na konci podle interní zvyklosti v O2.

Sloupec	Typ	Význam
load_dttm	DATETIME	Datum a čas načtení záznamu do původní DQ tabulky.
job_id	INT64	Identifikátor jobu, při kterém vznikla validační chyba.
table_name	STRING	Název zdrojové tabulky nebo objektu, ke kterému se záznam vztahuje.
file_name	STRING	Název souboru, ze kterého chybný řádek pochází.
line_no	INT64	Číslo řádku ve zdrojovém souboru.
error_cd	INT64	Kód validační chyby.
input_port	STRING	Vstupní port nebo pole, ve kterém byla chyba detekována.
sanitized_val	STRING	Očištěná nebo upravená hodnota z validačního procesu.
raw_line	STRING	Původní celý řádek ze zdrojového souboru.
collect_run_id	STRING	Jedinečný identifikátor běhu procedury.
collect_dttm	TIMESTAMP	Technický čas přenosu záznamu do centrální tabulky. Ukládá se v UTC.
source_project	STRING	Zdrojový BigQuery projekt.
source_dataset	STRING	Zdrojový dataset.

source_table	STRING	Zdrojová DQ tabulka, ze které byl záznam načten.
--------------	--------	--

5. Logika procedury sp_collect_dq10000

Procedura je navržena tak, aby běžela opakovaně, typicky přes Scheduled Query. V každém běhu si sama identifikuje vhodné zdrojové tabulky, provede jeden dynamický INSERT přes UNION ALL a následně uklidí pouze ty zdrojové tabulky, které byly bezpečně načteny.

Krok	Popis
1	Vygeneruje collect_run_id a collect_dttm pro celý běh.
2	Zapíše začátek běhu do dq10000_load_audit se statusem RUNNING.
3	Vytvoří dočasnou tabulku kandidátů z INFORMATION_SCHEMA.TABLE_STORAGE.
4	Kandidáti musí být BASE TABLE, obsahovat __dq10000__, být starší než 12 hodin a mít total_rows > 0.
5	Kandidáty zapíše do dq10000_load_table_audit se statusem CANDIDATE a drop_status NOT_DROPPED.
6	Pokud nejsou žádní kandidáti, ukončí běh se statusem NO_TABLES_TO_PROCESS.
7	Dynamicky sestaví jeden INSERT INTO daq_data.dq10000_all přes UNION ALL všech kandidátních tabulek.
8	Po insertu spočítá vložené řádky za collect_run_id.
9	V table auditu porovná source_rows proti loaded_row_count a označí tabulky jako LOADED nebo LOADED_ROW_DIFF.
10	DROP provede pouze u tabulek, které mají status LOADED a sedí počty řádků.
11	Po úspěšném DROP nastaví drop_status na DROPPED a doplní dropped_at.
12	Hlavní audit ukončí statusem DONE_WITH_DROP nebo DONE_WITH_DROP_WARNING.

6. Výběrová pravidla pro zdrojové tabulky

Kandidáti pro zpracování se vybírají z INFORMATION_SCHEMA.TABLE_STORAGE v regionu europe-west4.

```
FROM `o2czed1.region-europe-west4`.INFORMATION_SCHEMA.TABLE_STORAGE
WHERE table_schema = 'stg_lnd'
AND table_type = 'BASE TABLE'
AND REGEXP_CONTAINS(table_name, r'__dq10000__')
AND creation_time < TIMESTAMP_SUB(CURRENT_TIMESTAMP(), INTERVAL 12 HOUR)
AND total_rows > 0
```

Význam hlavních podmínek:

- table_schema = stg_lnd: pracujeme pouze s landing datasetem.
- table_type = BASE TABLE: zpracovávají se pouze fyzické tabulky, ne views nebo jiné objekty.

- REGEXP_CONTAINS(table_name, __dq10000__): berou se pouze DQ10000 tabulky.
- creation_time starší než 12 hodin: ochrana proti tabulkám, které mohou být ještě ve fázi plnění.
- total_rows > 0: do UNION ALL se berou jen tabulky, které obsahují data. Prázdné tabulky se do cíle neslévají.

7. Audit běhů

Hlavní auditní tabulka dq10000_load_audit obsahuje jeden řádek za každý běh procedury. Slouží pro rychlou kontrolu, zda procedura běžela, jak dopadla a kolik dat zpracovala.

Sloupec	Význam
collect_run_id	Identifikátor běhu procedury.
started_at	Čas začátku běhu v UTC.
finished_at	Čas konce běhu v UTC.
status	Stav běhu, například RUNNING, DONE_WITH_DROP, NO_TABLES_TO_PROCESS nebo ERROR.
source_table_count	Počet zdrojových tabulek vybraných pro zpracování.
inserted_row_count	Celkový počet vložených řádků do dq10000_all.
error_message	Chybová zpráva v případě neúspěchu.
created_by	Uživatel nebo účet, pod kterým byl běh spuštěn.

8. Detailní audit zdrojových tabulek

Detailní auditní tabulka dq10000_load_table_audit obsahuje jeden řádek za každou zdrojovou DQ tabulku v daném běhu. Je klíčová pro bezpečný DROP, protože umožňuje doložit, že každá tabulka byla načtena a že sedí počet řádků.

Sloupec	Význam
source_project/source_dataset/source_table	Identifikace zdrojové DQ tabulky.
source_rows	Počet řádků ve zdrojové tabulce podle TABLE_STORAGE v okamžiku výběru kandidátů.
loaded_row_count	Počet řádků skutečně načtených do dq10000_all pro danou source_table.
status	Stav načtení tabulky: CANDIDATE, LOADED, LOADED_ROW_DIFF nebo ERROR.
drop_status	Stav mazání: NOT_DROPPED, DROPPED nebo DROP_ERROR.
dropped_at	Čas úspěšného DROP TABLE.
drop_error_message	Chyba při DROP TABLE, pokud nastala.

9. Bezpečnostní princip pro DROP

DROP zdrojových tabulek se neprovádí přímo po nalezení kandidátů. Procedura nejdříve data načte, potom spočítá řádky v cílové tabulce a teprve pokud pro konkrétní tabulku platí source_rows = loaded_row_count, označí ji jako LOADED. DROP je povolen jen pro tyto tabulky.

DROP se provádí pouze pokud:
- status = 'LOADED'

```
- source_rows = loaded_row_count  
- drop_status = 'NOT_DROPPED'
```

Tím se minimalizuje riziko, že by byla smazána tabulka, jejíž data nebyla do cíle načtena správně.

10. Provozní spuštění

Proceduru lze spustit ručně nebo plánovaně přes BigQuery Scheduled Query.

Ruční spuštění:

```
CALL `o2czed1.daq_data.sp_collect_dq10000`0;
```

Scheduled Query obsahuje stejný příkaz CALL. V této fázi je vhodné plánování nastavit tak, aby odpovídalo provoznímu oknu, kdy už mají být DQ tabulky bezpečně dokončené. Pravidlo starší než 12 hodin slouží jako ochrana proti souběhu s ingestem.

11. Ověřovací dotazy po běhu

Kontrola posledních běhů:

```
SELECT  
  collect_run_id,  
  started_at,  
  finished_at,  
  status,  
  source_table_count,  
  inserted_row_count,  
  error_message,  
  created_by  
FROM `o2czed1.daq_data.dq10000_load_audit`  
ORDER BY started_at DESC  
LIMIT 10;
```

Kontrola detailu posledního běhu:

```
SELECT  
  collect_run_id,  
  source_table,  
  source_rows,  
  loaded_row_count,  
  source_rows - loaded_row_count AS diff_rows,  
  status,  
  drop_status,  
  dropped_at,  
  drop_error_message  
FROM `o2czed1.daq_data.dq10000_load_table_audit`  
WHERE collect_run_id = (  
  SELECT collect_run_id  
  FROM `o2czed1.daq_data.dq10000_load_audit`  
  ORDER BY started_at DESC  
  LIMIT 1
```

```
)  
ORDER BY source_rows DESC;
```

Kontrola, že v landing vrstvě nezůstaly neprázdné DQ tabulky:

```
SELECT  
  COUNT(*) AS remaining_non_empty_dq_tables  
FROM `o2czed1.region-europe-west4`.INFORMATION_SCHEMA.TABLE_STORAGE  
WHERE table_schema = 'stg_lnd'  
  AND table_type = 'BASE TABLE'  
  AND REGEXP_CONTAINS(table_name, r'__dq10000__')  
  AND creation_time < TIMESTAMP_SUB(CURRENT_TIMESTAMP(), INTERVAL 12 HOUR)  
  AND total_rows > 0;
```

Poznámka: INFORMATION_SCHEMA.TABLE_STORAGE může mít po DROPu krátké zpoždění v propsání metadat. Pro rychlou kontrolu existence tabulek lze použít také INFORMATION_SCHEMA.TABLES.

```
SELECT  
  COUNT(*) AS remaining_dq_tables  
FROM `o2czed1.stg_lnd`.INFORMATION_SCHEMA.TABLES  
WHERE table_type = 'BASE TABLE'  
  AND REGEXP_CONTAINS(table_name, r'__dq10000__');
```

12. Očekávané stavy

Situace	Očekávaný stav
Úspěšný běh s načtením a DROPem	Run audit: DONE_WITH_DROP. Table audit: LOADED + DROPPED.
Nejsou žádné vhodné tabulky	Run audit: NO_TABLES_TO_PROCESS. Inserted_row_count = 0.
Nesedí počty řádků	Table audit: LOADED_ROW_DIFF. Tabulka se nedropne.
Chyba při DROP	Table audit: DROP_ERROR a vyplněná drop_error_message.
Obecná chyba procedury	Run audit: ERROR a vyplněná error_message.

13. Co jsme ověřili v testu

- Dataset stg_lnd obsahoval DQ tabulky s patternem __dq10000__.
- Všechny ověřené DQ tabulky měly stejnou strukturu 9 sloupců.
- Neprázdných kandidátních tabulek bylo 35.
- UNION ALL přes kandidátní tabulky fungoval a v testu vracel 21 703 337 řádků.
- Data byla úspěšně vložena do daq_data.dq10000_all.
- Audit běhu i detailní audit tabulek funguje.
- Po doplnění DROP logiky byly zdrojové tabulky úspěšně smazány a v detailním auditu označeny jako DROPPED.

14. Následné kroky

- Dohodnout finální plán spuštění Scheduled Query v provozním režimu.

- Ověřit, pod jakým účtem nebo service accountem má Scheduled Query běžet.
- Doplnit provozní monitoring nad dq10000_load_audit a dq10000_load_table_audit.
- Rozhodnout, zda a jak uklízet historické prázdné __dq10000__ tabulky v stg_lnd. Aktuální merge proces je záměrně nebere do UNION ALL. Prázdné tabulky lze řešit samostatným cleanup procesem.
- Zvážit doplnění notifikace při stavu ERROR, DROP_ERROR nebo DONE_WITH_DROP_WARNING.
- Před převodem do dalšího prostředí upravit názvy projektů, datasetů, region a případně název service accountu.

15. Doporučení pro kolegy

Proces je navržen tak, aby většina logiky zůstala přímo v BigQuery. To je vhodné, protože se jedná primárně o SQL operaci nad BigQuery tabulkami. Není potřeba externí Python, Windows Task Scheduler ani přenos dat mimo BigQuery. Python nebo externí scheduler by dal smysl až ve chvíli, kdy by bylo potřeba volat externí systémy, posílat e-maily, pracovat se soubory mimo BigQuery nebo řešit složitější retry logiku.